

Parallel and Concurrent programming

1 Obstruction-Free atomic snapshot implementation on MRMW registers

Global Variables

- **Input/Output Files:**
 - `inputFileName`: Name of the input file for fetching input details.
 - `outputFileName`: Name of the output file for writing output into the file.
- **Configuration Parameters:**
 - `inputFileValues`: Vector to store the parameters fetched from the input file.
 - `numberOfWriterThreads`: Number of writer threads present.
 - `numberOfReaderThreads`: Number of snapshot threads present.
 - `sizeOfRegister`: Size of the shared register present.
 - `waitTimeForNextWrite`: Delay between the write operations.
 - `waitTimeForNextRead`: Delay between the read operations.
 - `totalNumberOfSnapshots`: Total snapshots to be collected.
- **Mutexes:**
 - `printMutex`: Protects printing to the output file using lock.
 - `registerMutex`: Protects the shared register during updates using lock.

Data Structures

- **StampedValue:**
 - Holds a value and a timestamp (as an atomic integer).
 - Constructors for initialization.
- **registerArray:**
 - A vector of `shared_ptr<StampedValue>` representing the shared register.

Functions

- `readingInputFile()`: Reads parameters from the input file into `inputFileValues`.
- `fetchingInputDetails()`: Extracts configuration parameters from `inputFileValues`.
- `getCurrentSystemTime()`: Returns the current system in the specified format.
- `update(int index, int value)`:
 - Updates the specified index in the `registerArray` with a new value and increments its timestamp.
 - Uses `registerMutex` used for thread-safe updates.
- `writerThreadOperation(atomic<bool> term, int processId, ofstream outputFile)`:
 - It continuously writes random values to the register.
 - Generates a random index and value, calls `update()`, and logs the operation.
- `collect()`: Takes a snapshot of the current state of the `registerArray` while holding `registerMutex`.
- `snapshotThreadOperation(atomic<bool> term, int snapshotId, ofstream outputFile)`:
 - Collects snapshots of the register.
 - Compares old and new snapshots to detect changes.
 - Logs completed snapshots to the output file.

Main Function

- Initializes the random seed.
- Reads parameters from the input file.
- Allocates and initializes the `registerArray`.
- Opens the output file for logging.
- Launches writer threads and snapshot threads using the STL `thread` library.
- Sleeps for a specified duration to allow threads to operate.
- Signals threads to terminate and waits for them to finish.
- Closes the output file.

Flow

1. **Initialization:** Parameters are read from the input file and stored in global variables.
2. **Thread Creation:** Writer threads are launched to update the register with random values. Snapshot threads are launched to periodically take snapshots of the register.
3. **Concurrent Operations:** Writers and snapshot threads operate concurrently, with proper synchronization to prevent data races.
4. **Termination:** After a set period, all threads are signaled to terminate, ensuring all operations are completed before exiting.

Key Factors

- **Thread Safety:** Mutexes are employed to ensure that access to the shared register and output file is thread-safe.
- **Atomic Operations:** The `atomic<int>` type is used for the timestamp to ensure that it can be safely accessed and modified across threads without additional locking.
- **Output Synchronization:** Mutexes prevent interleaved output from different threads, maintaining readability in the log file.

2 Wait-Free atomic snapshot implementation on MRMW registers

Global Variables

- **Input/Output Files:**
 - `inputFileName`: Name of the input file for parameters.
 - `outputFileName`: Name of the output file for logging results.
- **Configuration Parameters:**
 - `inputFileValues`: Vector to store parameters read from the input file.
 - `numberOfWriterThreads`: Number of writer threads.
 - `numberOfReaderThreads`: Number of snapshot threads.
 - `sizeOfRegister`: Size of the shared register.
 - `waitTimeForNextWrite`: Delay between write operations.
 - `waitTimeForNextRead`: Delay between read operations.
 - `totalNumberOfSnapshots`: Total snapshots to collect.
- **Mutexes:**
 - `printMutex`: Protects output to the log file.
 - `registerMutex`: Protects access to the shared register during updates.

Data Structures

- **StampedValue:**
 - Holds a register value and an atomic timestamp.
 - Constructors for initialization and copy construction.
- **registerArray:**
 - A vector of `shared_ptr<StampedValue>` representing the shared register.

Functions

- `readingInputFile()`: Reads parameters from the specified input file into `inputFileValues`.
- `fetchingInputDetails()`: Extracts configuration parameters from `inputFileValues`.
- `getCurrentSystemTime()`: Returns the current system time as a formatted string (HH:MM:SS:ms).
- `update(int index, int value)`:
 - Updates the specified index in the `registerArray` with a new value and increments its timestamp.
 - Uses `registerMutex` to ensure thread-safe updates.
- `collect()`: Takes a snapshot of the current state of the `registerArray` while holding `registerMutex`.
- `writerThreadOperation(atomic<bool> term, int processId, ofstream outputFile)`:
 - Continuously writes random values to the register.
 - Generates a random index and value, calls `update()`, and logs the operation to the output file.
- `snapshotThreadOperationWaitFree(atomic<bool> term, int snapshotId, ofstream outputFile)`:
 - Collects snapshots of the register in a wait-free manner.
 - Compares old and new snapshots to check for consistency.
 - Uses a `moved` vector to track which registers have been processed.
 - Logs completed snapshots to the output file.

Main Function

- Initializes the random seed.
- Reads parameters from the input file.
- Allocates and initializes the `registerArray`.
- Opens the output file for logging.
- Launches writer threads to update the register.

- Launches snapshot threads to periodically collect snapshots.
- Sleeps for a specified duration to allow threads to operate.
- Signals all threads to terminate after the sleep period.
- Waits for all threads to finish and closes the output file.

Flow

1. **Initialization:** Parameters are read from the input file and stored in global variables.
2. **Thread Creation:** Writer threads are created to update the register with random values. Snapshot threads are created to periodically collect snapshots of the register.
3. **Concurrent Operations:** Writers and snapshot threads operate concurrently, with synchronization to prevent data races. Snapshot threads implement a wait-free approach to ensure consistency while collecting data.
4. **Termination:** After a set period, all threads are signaled to terminate. The main thread waits for all threads to complete their operations before exiting.

Key Factors

- **Thread Safety:** Mutexes are used to ensure that access to the shared register and output file is thread-safe.
- **Atomic Operations:** The use of `atomic<int>` for timestamps ensures safe access and modification across threads without additional locking.
- **Wait-Free Snapshotting:** The snapshot operation allows for the collection of register states without blocking other threads, increasing efficiency and reducing latency.
- **Output Synchronization:** The `printMutex` prevents interleaved output from different threads, ensuring that log entries are coherent and readable.

3 Graphs

3.1 Graph 1 - Average-case Scalability

In this graph, the Y-axis will represent two values, the average time taken by Obstruction-Free and Wait-Free snapshot algorithms in each of the following cases update threads, scan threads, all threads (update + scan). The X-axis will depict the number of threads. The number of threads will vary from 4 to 64 in powers of 2. To maintain consistency in the experiment, all other parameters will remain constant as follows: $nw/ns = 4$, $M = 40$, $w = 0.5$, $s = 0.5$, $k = 5$.

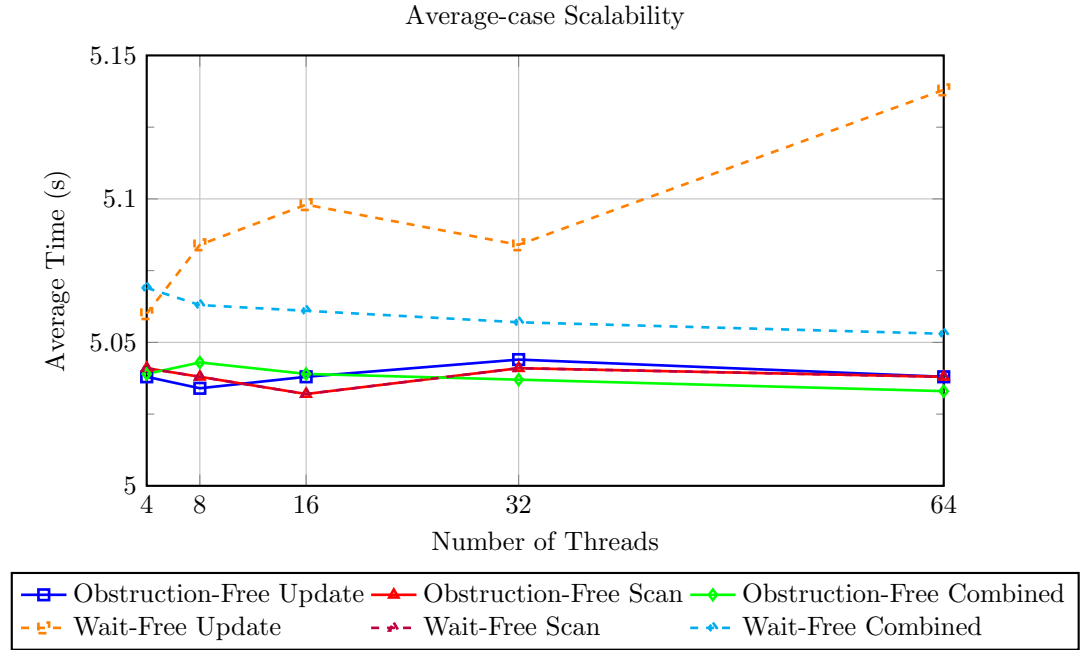


Figure 1: Average-case Scalability for Obstruction-Free and Wait-Free Snapshot Algorithms

Conclusion - The data suggests that both Obstruction-Free and Wait-Free algorithms exhibit robust scalability in average-case scenarios, making them suitable for multi-threaded applications where performance needs to be maintained as thread counts increase.

3.2 Graph 2 - Worst-case Scalability

In this graph, the Y-axis will represent two values, the worst-case time taken by Obstruction-Free and Wait-Free snapshot algorithms in each of the following cases update threads, scan threads, all threads (update + scan). The X-axis will depict the number of threads. The number of threads will vary from 4 to 64 in powers of 2. To maintain consistency in the experiment, all other parameters will remain constant as follows: $nw/ns = 4$, $M = 40$, $w = 0.5$, $s = 0.5$, $k = 5$.

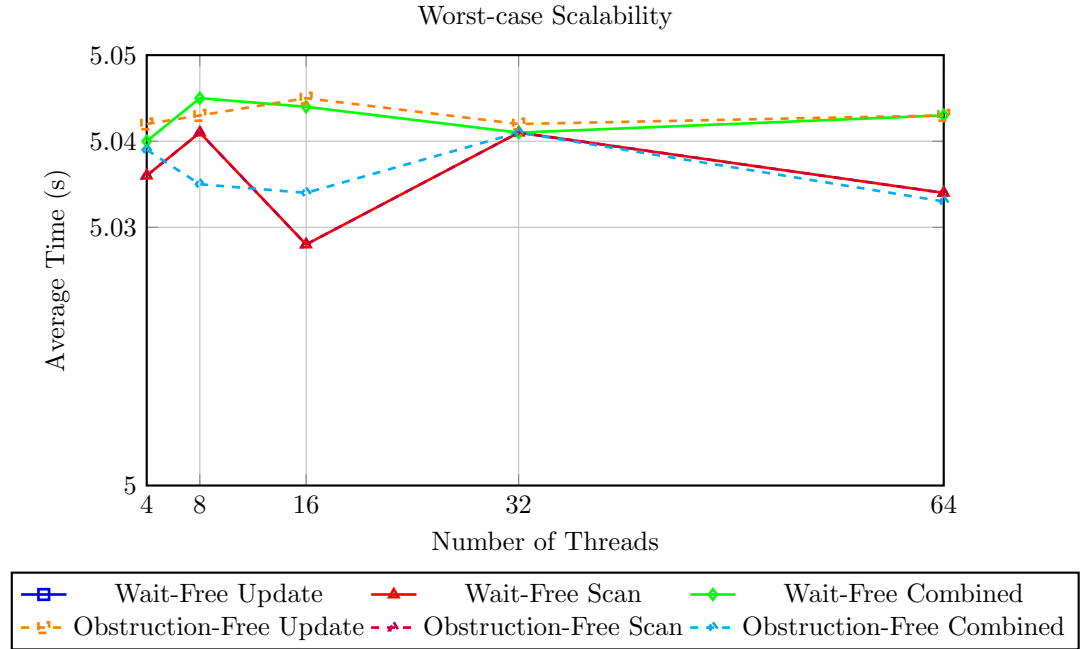


Figure 2: Worst-case Scalability for Obstruction-Free and Wait-Free Snapshot Algorithms

Conclusion - Both algorithms exhibit strong scalability in worst-case scenarios, although the Obstruction-Free algorithm tends to outperform the Wait-Free algorithm in terms of update times. This highlights its potential suitability for applications where worst-case performance is critical.

3.3 Graph 3 - Impact of update operation on Scan in the average-case

In this graph, the Y-axis will represent two values, the average-case time taken by Obstruction-Free and Wait-Free snapshot algorithms in each of the following cases update threads, scan threads, all threads (update + scan). The X-axis will depict the ratio of n_w/n_s , i.e. the ratio of the number of writer threads to the number of snapshot threads. It will specifically consist of the following 6 values: 10, 8, 6, 4, 2, 1. Have the remaining parameters as: $n_s = 4$, $M = 20$, $w = 0.5$, $s = 0.5$, $k = 64$. Given the value of n_s as 4, based on the ratio of n_w/n_s , the value of n_w can be determined.

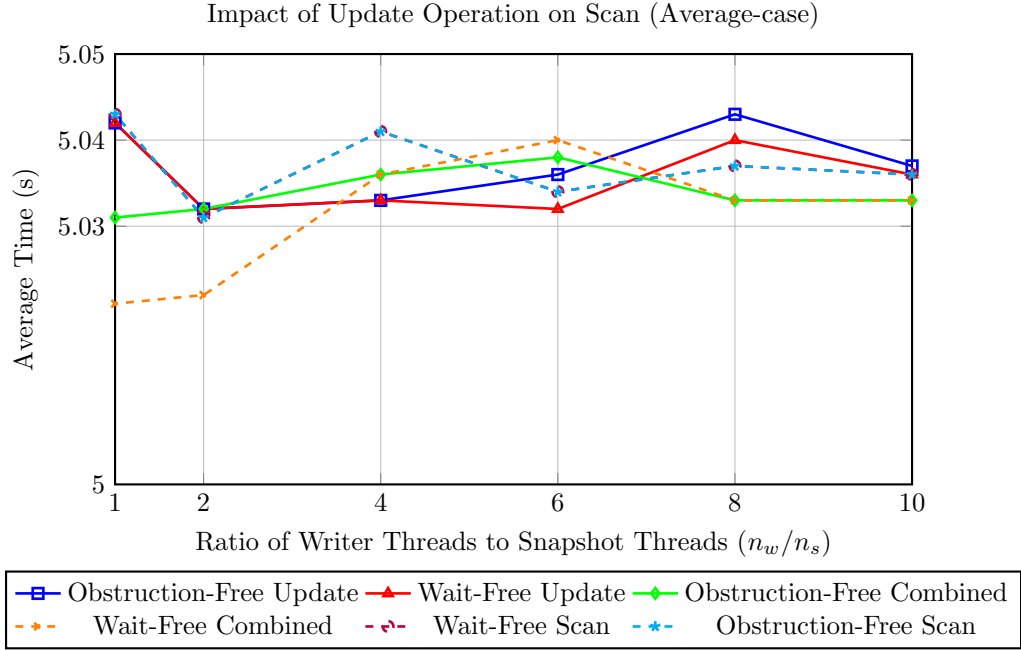


Figure 3: Impact of Update Operation on Scan in Average-case for Obstruction-Free and Wait-Free Snapshot Algorithms

Conclusion - The data suggests that the update operations have a minimal negative impact on scan performance in average-case scenarios for both algorithms, allowing for efficient multi-threaded processing without significant delays.

3.4 Graph 4 - Impact of update operation on Scan in the worst-case

In this graph, the Y-axis will represent two values, the worst-case time taken by Obstruction-Free and Wait-Free snapshot algorithms in each of the following cases: (1) update threads, (2) scan threads, (3) all threads (update + scan). The X-axis will depict the ratio of n_w/n_s , i.e. the ratio of the number of writer threads to the number of snapshot threads. It will specifically consist of the following 6 values: 10, 8, 6, 4, 2, 1. Have the remaining parameters as: $n_s = 4$, $M = 20$, $w = 0.5$, $s = 0.5$, $k = 64$. Given the value of n_s as 4, based on the ratio of n_w/n_s , the value of n_w can be determined.

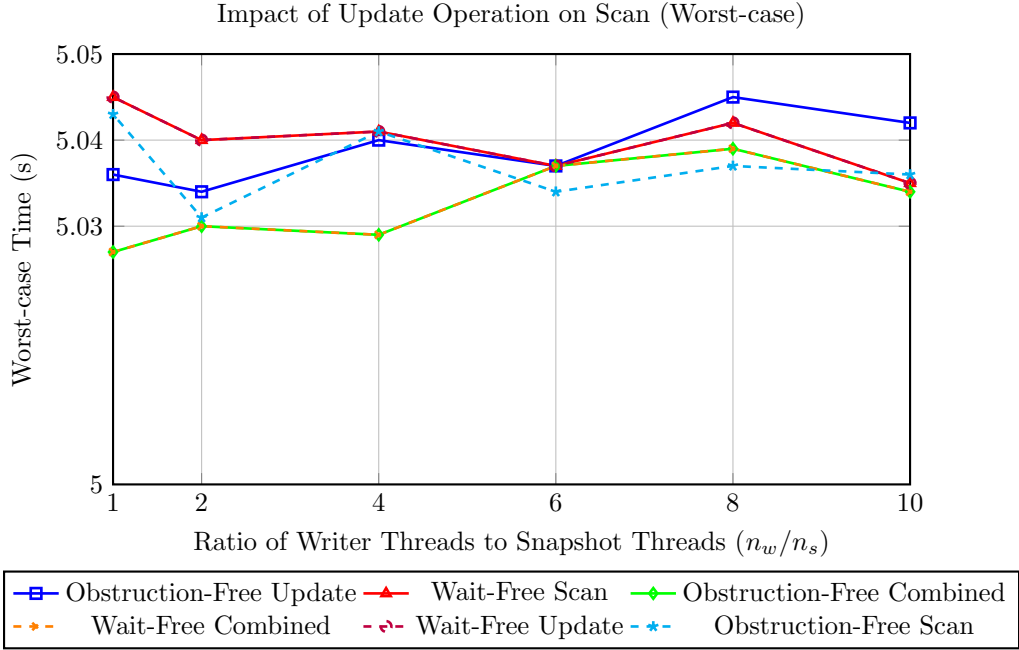


Figure 4: Impact of Update Operation on Scan in Worst-case for Obstruction-Free and Wait-Free Snapshot Algorithms

Conclusion - While the worst-case performance for both algorithms is generally stable, the Obstruction-Free algorithm shows an advantage in handling updates compared to the Wait-Free algorithm. This suggests that the Obstruction-Free algorithm may be more effective in scenarios where high update rates are expected, while still allowing for reliable scan operations.